

S&P 500 Factor Investing

A multi-strategy research project: quality, value, momentum and trend

Group members

Umut Bülbül	2022300318
Sıla Tanrıverdioğlu	2021300222
Sude Abalı	2021300093
Ömer Faruk Yıldız	2021300162

Reviewed paper: Han, Zhou and Zhu (2016)

A trend factor that combines information across multiple moving-average horizons.

Paper idea

- Uses normalized moving averages at 11 horizons: 3 to 1000 trading days.
- Each month estimates a cross-sectional regression of next-month returns on the MA signals.
- Forms a long-short factor from predicted returns: long high expected return, short low expected return.
- In the paper, the trend factor improves on standalone reversal and momentum signals.

Project adaptation

- The project is long-only and S&P 500-focused, so it is not a full paper replication.
- Improved 6 is the closest paper-faithful variant: HZZ stock-level cross-sectional trend.
- Base and Improved 1-5/7/8 use the assignment-driven index-regression trend on $\wedge$ GSPC.
- This distinction explains why Imp 4 and Imp 6 must be compared explicitly.

Data universe

Raw input	Source	Rows	Span	Role
Daily prices	Tiingo	2.29M	2006-2026	OHLCV + adjusted prices
Daily fundamentals	Tiingo	2.30M	2006-2026	P/E, P/B, PEG, market cap
Statement fundamentals	Tiingo	3.46M	2005-2026	ROE via date_available
S&P 500 constituents	Wikipedia snapshot	503	current	Universe definition
Index benchmark	Yahoo ^GSPC	5,030	2006-2026	Benchmark + trend input

Largest data weakness

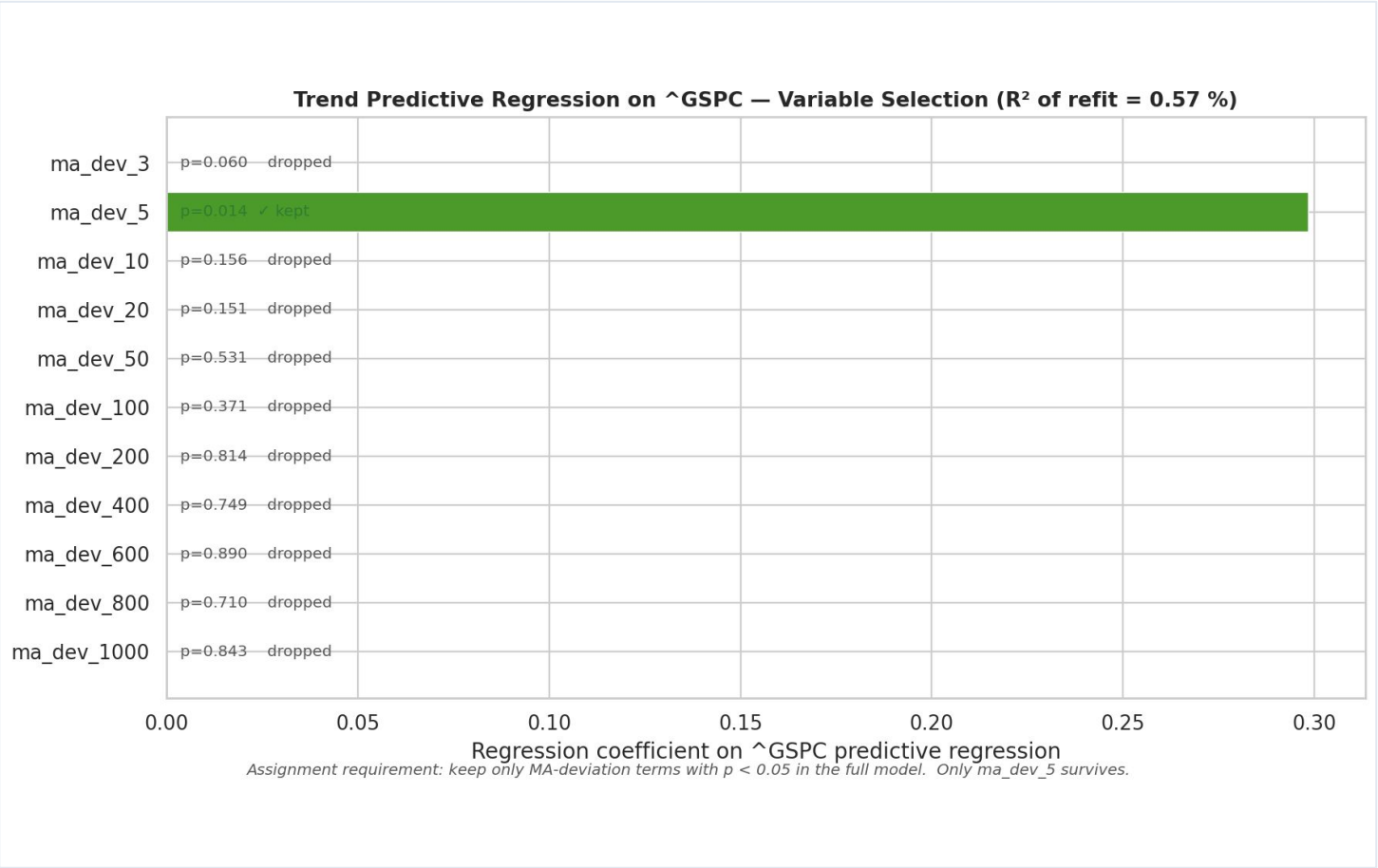
- Universe is current S&P 500 constituents only.
- Removed or failed firms are absent.
- Recent additions have backfilled histories before index membership.
- Expected bias: Sharpe too high, drawdown too low.

Factor construction and trend-variable selection

Four signals are standardized monthly; trend is the methodological fork.

Factor pipeline

- ROE: quality signal from point-in-time statements.
- -P/E: value signal, where cheaper stocks score higher.
- 12-month momentum: price return from t-12 to t.
- Trend: moving-average signal from either ^GSPC regression or HZZ cross-sectional regression.
- All signals are lagged one month, winsorized at 1/99 pct, then z-scored cross-sectionally.



Factor diagnostics: trend carries most of the standalone signal

The four-factor composite is not equally supported by the data.



Interpretation: trend is the most consistent positive factor. ROE and P/E are weak in isolation; their role is closer to diversification than standalone alpha.

Common evaluation window and strategy scoreboard

Vector metrics over 2016-05-31 to 2026-05-31; Base shown only as a biased assignment baseline.

Strategy	Sharpe	Ann. Ret.	Max DD	Cum. Ret.	Final \$	MC p	Beats ^GSPC?
Base*	1.20	7.23%	7.3%	103.2%	\$2.03M	0.013	No
Imp 1	1.18	13.12%	15.8%	251.0%	\$3.51M	0.019	No
Imp 2	1.05	10.46%	13.0%	172.1%	\$2.72M	0.068	No
Imp 3	0.97	10.18%	14.1%	163.5%	\$2.64M	0.157	No
Imp 4	1.15	10.48%	7.4%	175.2%	\$2.75M	0.088	No
Imp 5	0.93	7.96%	11.2%	114.8%	\$2.15M	0.099	No
Imp 6	1.02	7.35%	8.0%	104.1%	\$2.04M	0.290	No
Imp 8	1.03	13.71%	10.9%	263.3%	\$3.63M	0.306	No

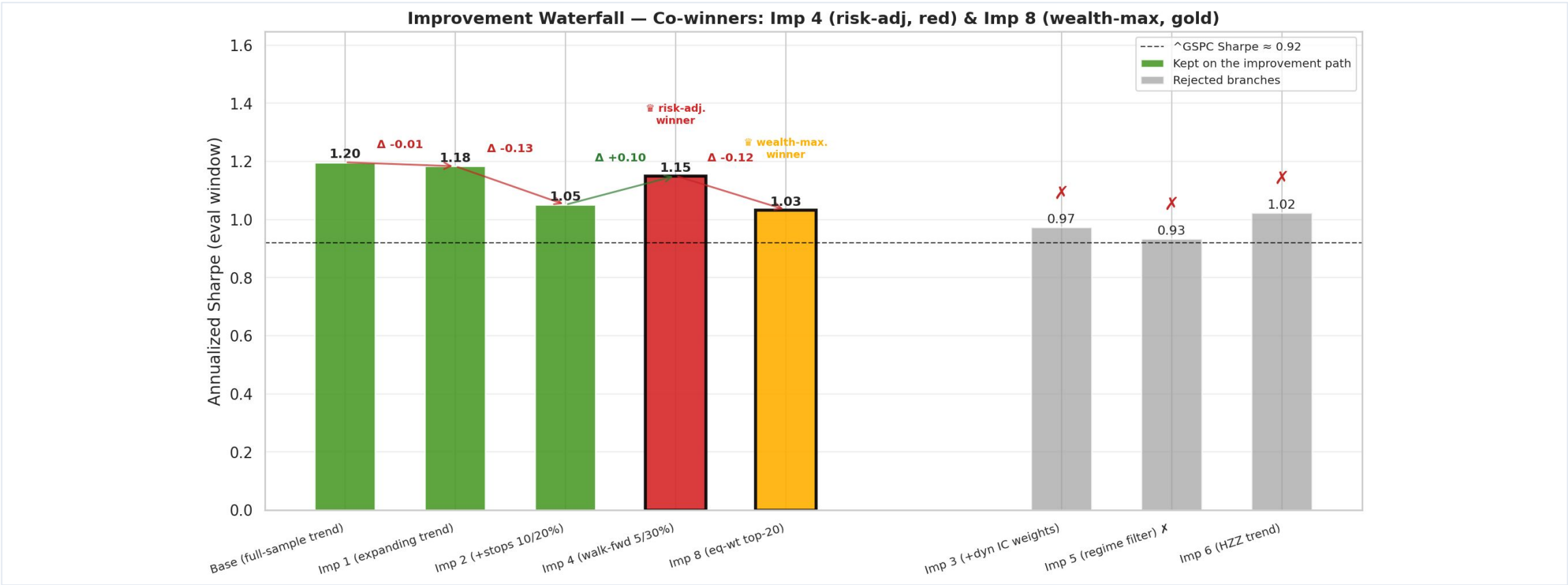
Risk-adjusted candidate

Imp 4: best Sharpe/drawdown balance among honest variants.

Wealth candidate

Imp 8: highest cumulative return and final equity, but lower Sharpe and worse drawdown.

Strategy ladder: one design change at a time



How the strategy is coded: declarative specifications

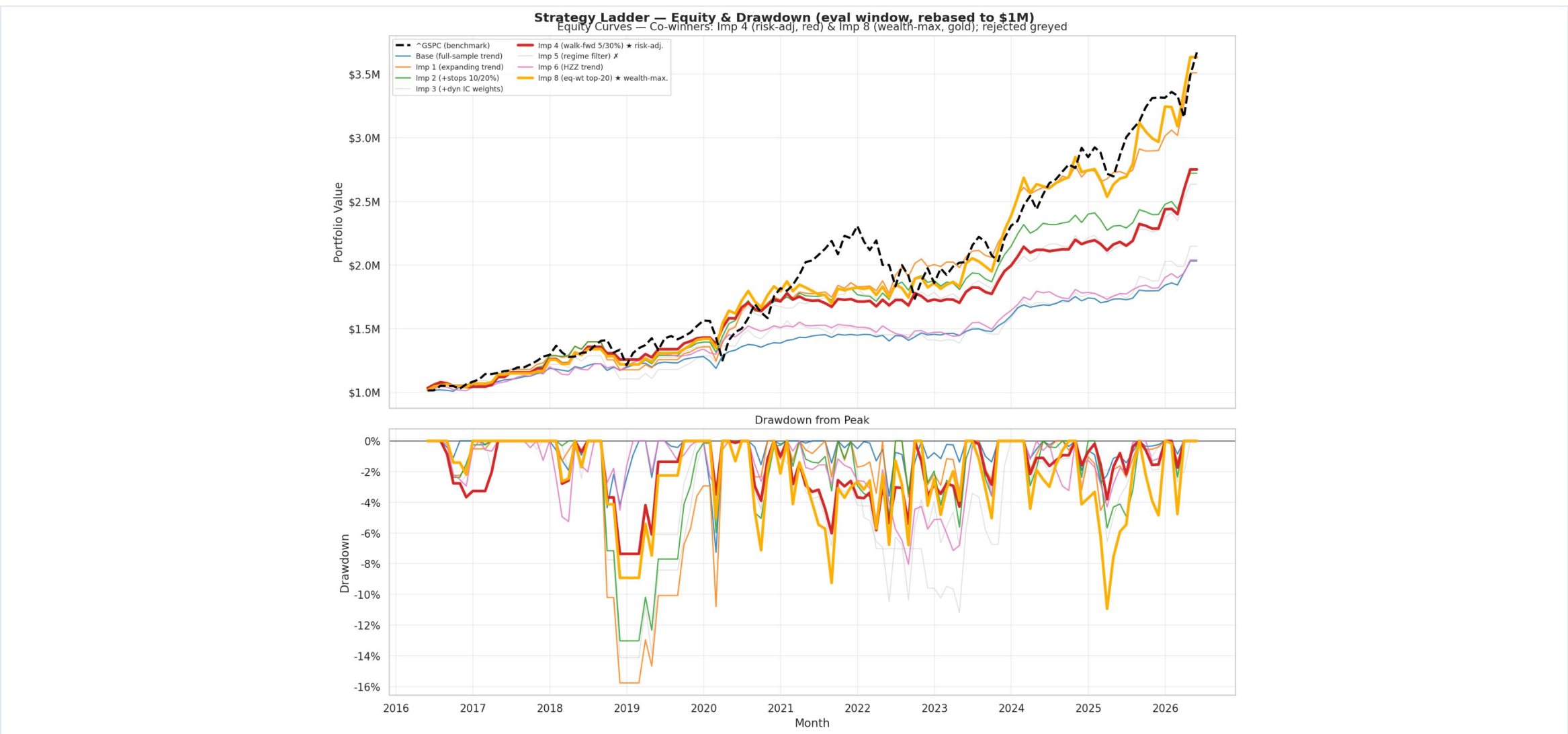
Each variant is a StrategySpec; runner files differ only by the intended experiment.

```
@dataclass(frozen=True)
class StrategySpec:
    name: str
    weights: dict[str, float]
    top_n: int = 10
    regime_filter: bool = False
    stop_loss: float | None = None
    take_profit: float | None = None
    trend_col: str = "trend_z"
    weight_method: str = "static"
    sizing_method: str = "fixed_cash"
    sizing_target_pct: float = 0.10
```

Variant	Code-level change
Base	top_n=10; trend_col="trend_z"; no stop/take
Imp 1	trend_col="trend_expanding_z"; no look-ahead
Imp 2	adds 10% stop / 20% take
Imp 3	weight_method="rolling_rank_ic"
Imp 4	top-10; 5% stop / 30% take
Imp 5	Imp 4 + regime_filter=True
Imp 6	trend_col="trend_hzz_z"
Imp 8	top_n=20; percent_of_equity; 5% each

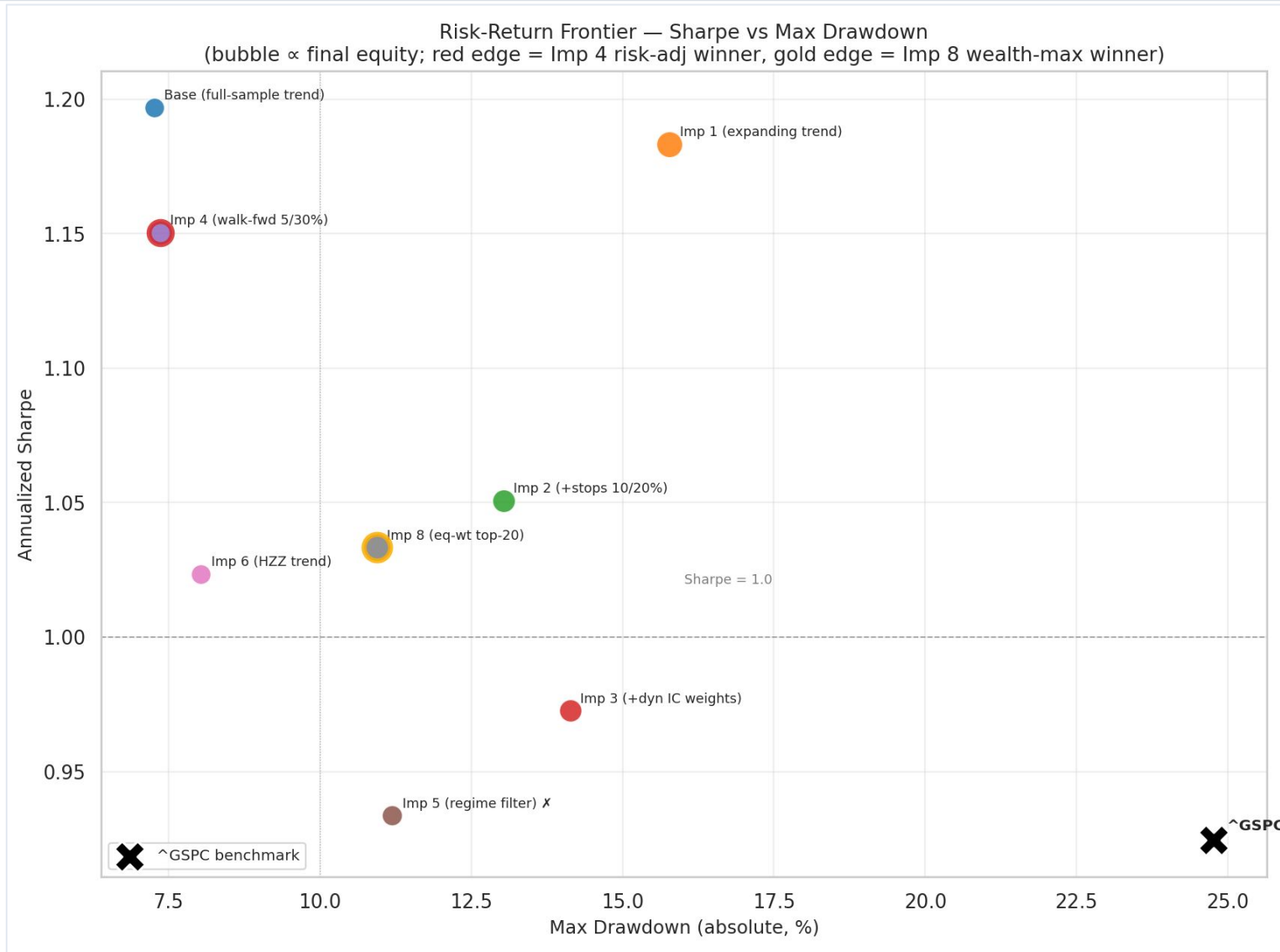
Performance paths: strategy ladder vs benchmark

Equity and drawdown reveal the risk-return trade-off better than a single Sharpe number.



Risk-return frontier: where each variant sits

Sharpe vs max drawdown clarifies the winner criterion.

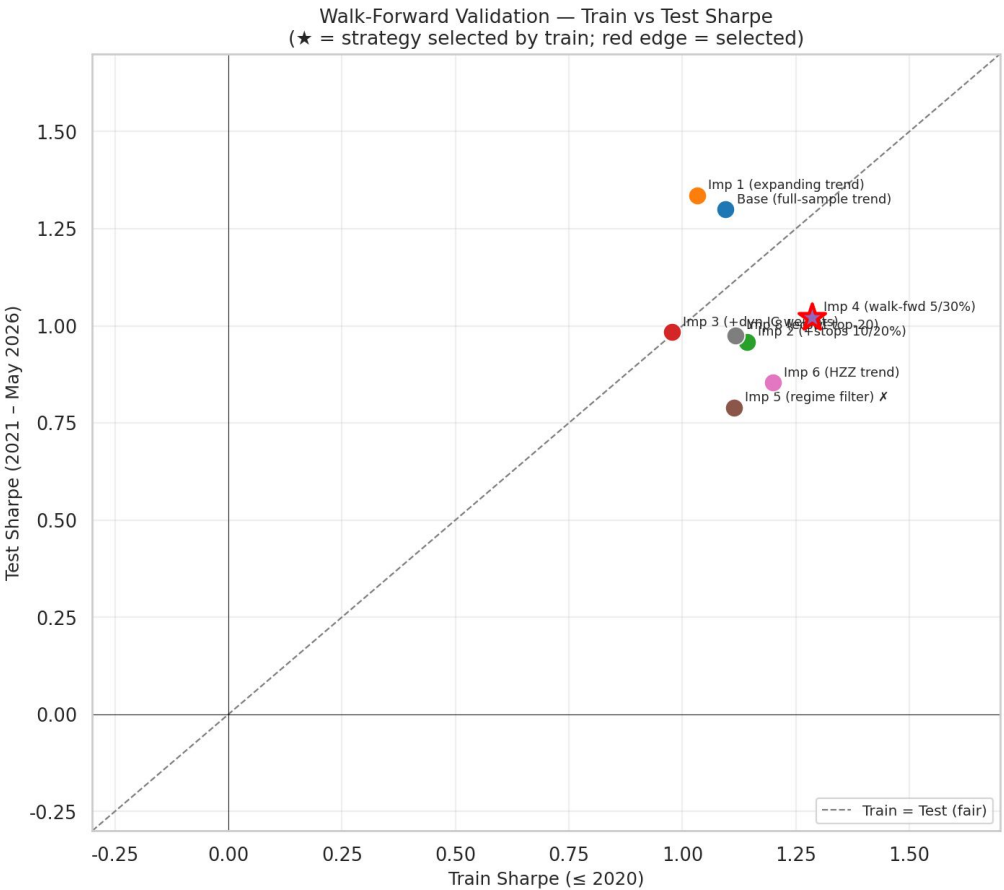


- Higher Sharpe is better.
- Lower absolute drawdown is better.
- Imp 4 is near the efficient corner among honest variants.
- Imp 8 gives up Sharpe and drawdown control in exchange for compounding more capital.
- The ^GSPC point has much worse drawdown and lower Sharpe in this window.

Walk-forward validation: train selection vs test performance

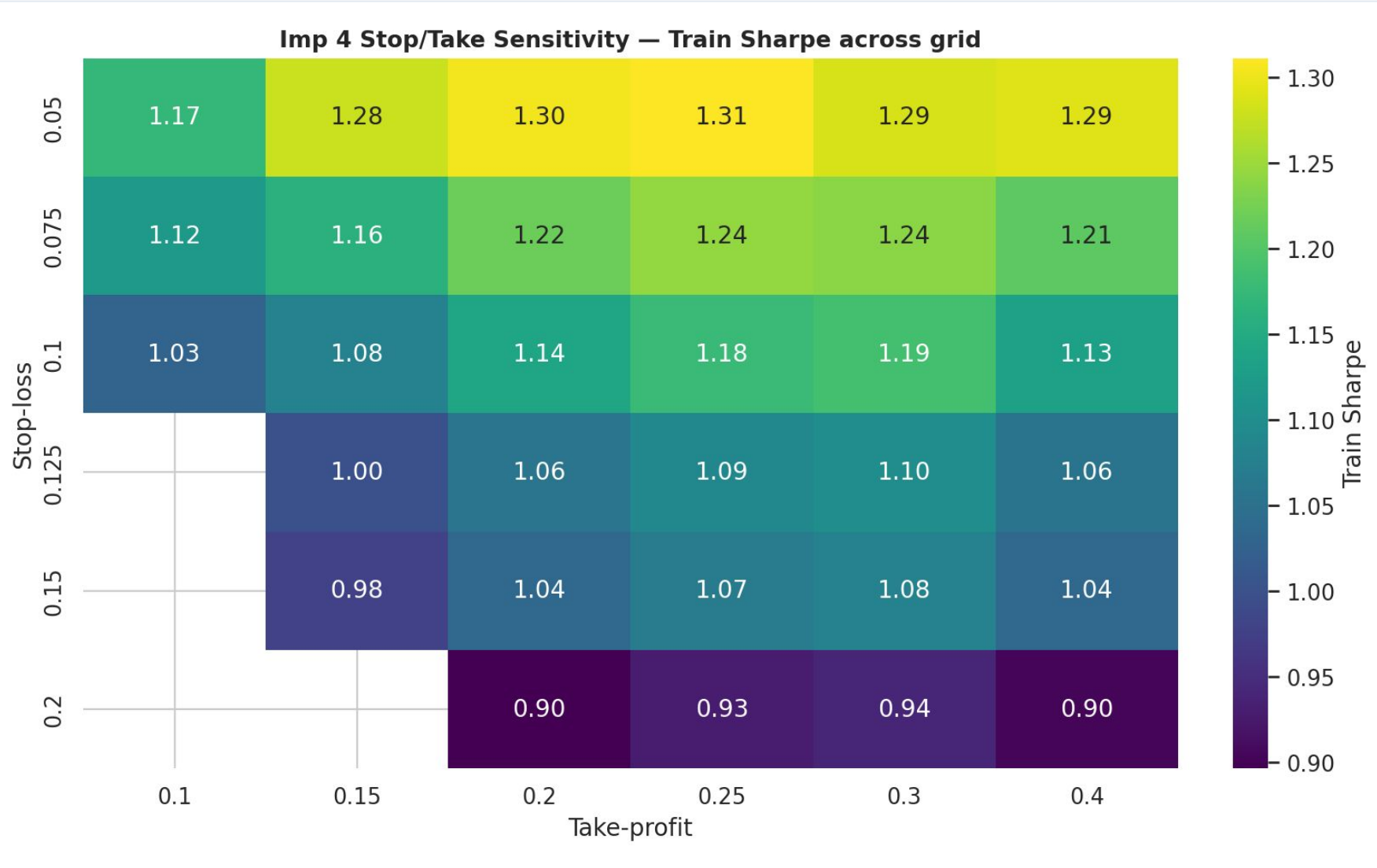
Train through 2020-12; test from 2021 to May 2026.

Strategy	Train Sharpe	Test Sharpe	Test CumRet	Test MaxDD	Selected by train?
Imp 4 (walk-fwd 5/30%)	1.29	1.02	60.4%	6.0%	Yes
Imp 6 (HZZ trend)	1.20	0.85	35.1%	8.0%	
Imp 2 (+stops 10/20%)	1.14	0.96	58.5%	5.8%	
Imp 8 (eq-wt top-20)	1.12	0.98	102.5%	10.9%	
Imp 5 (regime filter)	1.11	0.79	43.4%	11.2%	
Base*	1.10	1.30	46.5%	3.6%	
Imp 1 (expanding trend)	1.03	1.34	103.1%	5.2%	
Imp 3 (+dyn IC weights)	0.98	0.98	57.4%	10.5%	



Why Improved 4: walk-forward stop/take selection

The 5% stop and 30% take-profit were selected on training data, not after seeing the full sample.



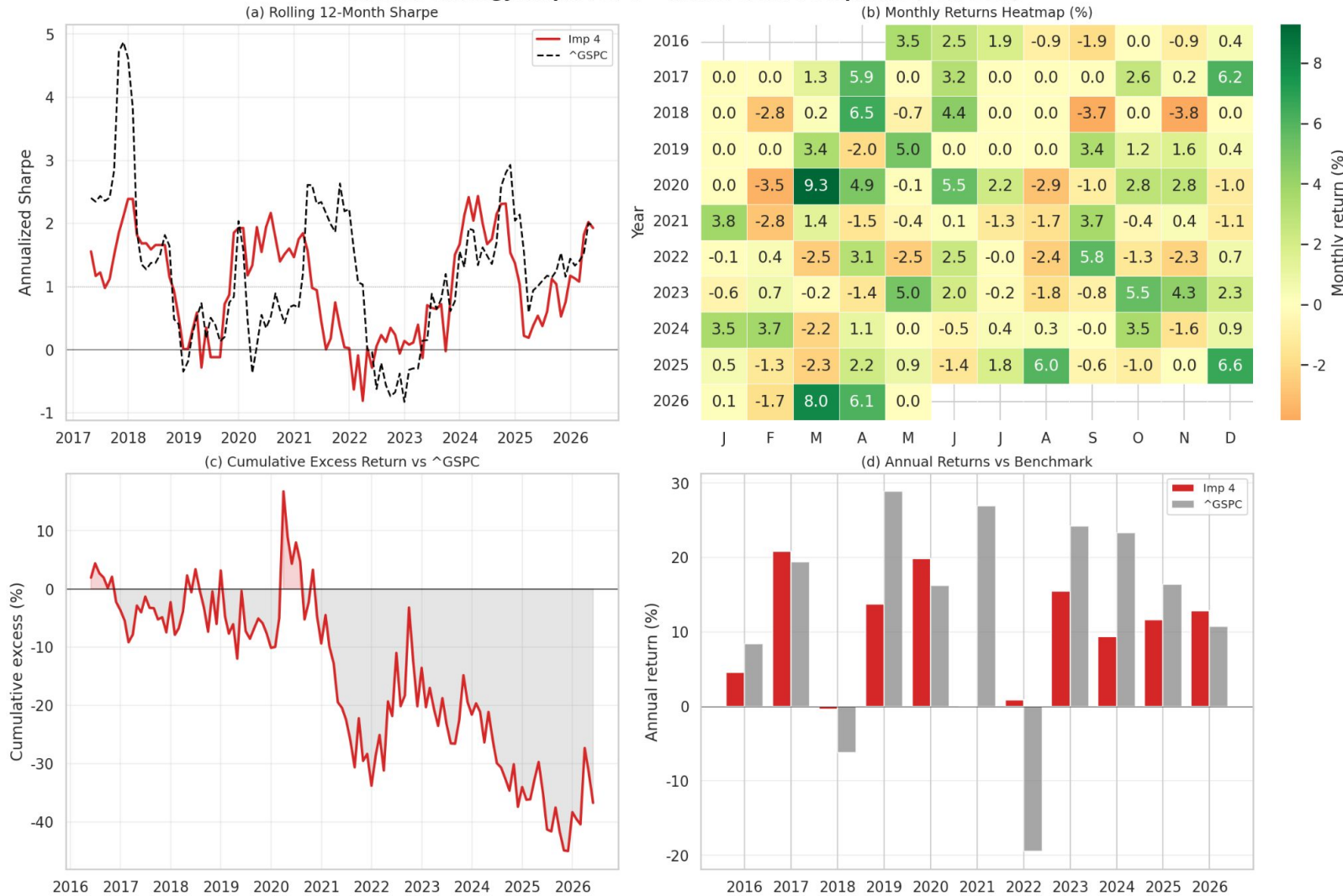
Selected setting

- Stop-loss: 5%
- Take-profit: 30%
- Chosen using train data up to 2020-12.
- The selected pair improves drawdown control without sacrificing much return.
- The grid search itself still counts as model selection.

Imp 4 is the risk-managed winner, not necessarily the highest terminal-wealth variant.

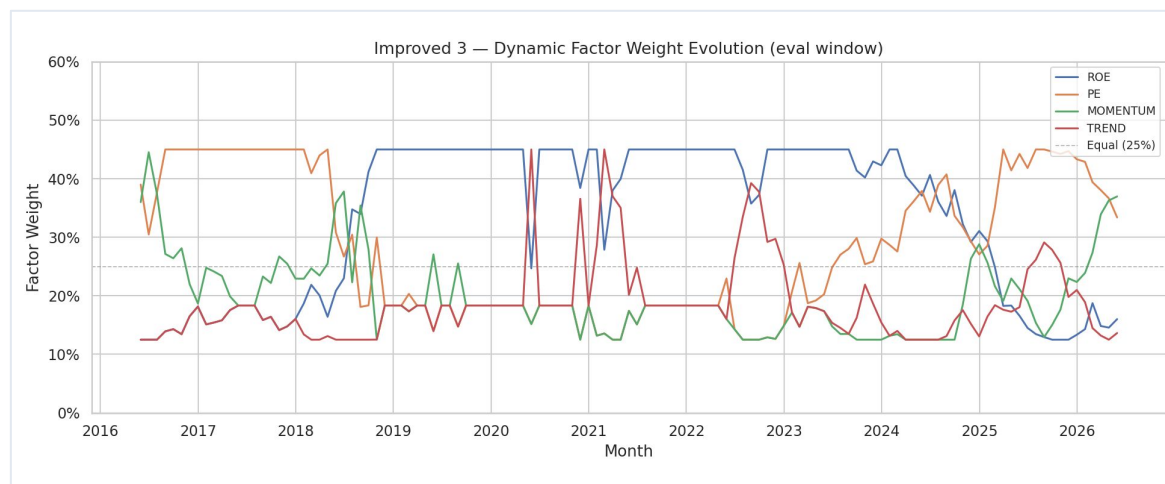
Selected strategy: Improved 4

Selected Strategy: Improved 4 — Walk-Forward Stop/Take (5% / 30%)



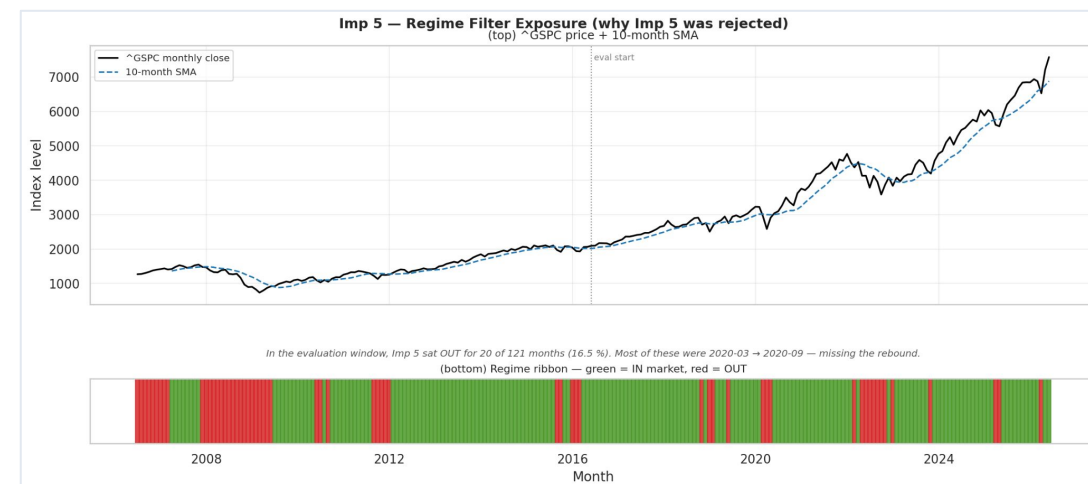
Rejected branches: complexity and timing did not automatically help

Two failed ideas are useful because they explain what not to do.



Imp 3: dynamic factor weights

Rolling IC weights became unstable and often hit caps. The added complexity did not improve the risk-return profile.



Imp 5: 10-month SMA regime filter

The filter sat out for 20 of 121 eval months and missed key rebound exposure, especially in 2020.

Lesson: more adaptive rules can reduce robustness unless they add stable out-of-sample information.

Improved 4 vs Improved 6: why the paper-faithful version did not win

Imp 6 implements HZZ cross-sectional trend, but Imp 4 dominates most project metrics.

Metric	Imp 4	Imp 6	Winner
Annualized Sharpe	1.15	1.02	Imp 4
Annualized Return	10.48%	7.35%	Imp 4
Max Drawdown	7.4%	8.0%	Imp 4
Cumulative Return	175.2%	104.1%	Imp 4
Final Equity	\$2.75M	\$2.04M	Imp 4
# Trades	687	1072	Imp 4
Avg hold days	22.1	25.0	Imp 6
Hit-rate	48.8%	53.7%	Imp 6
MC p-value raw	0.088	0.290	Imp 4
Romano-Wolf vs ^GSPC	No	No	Tie

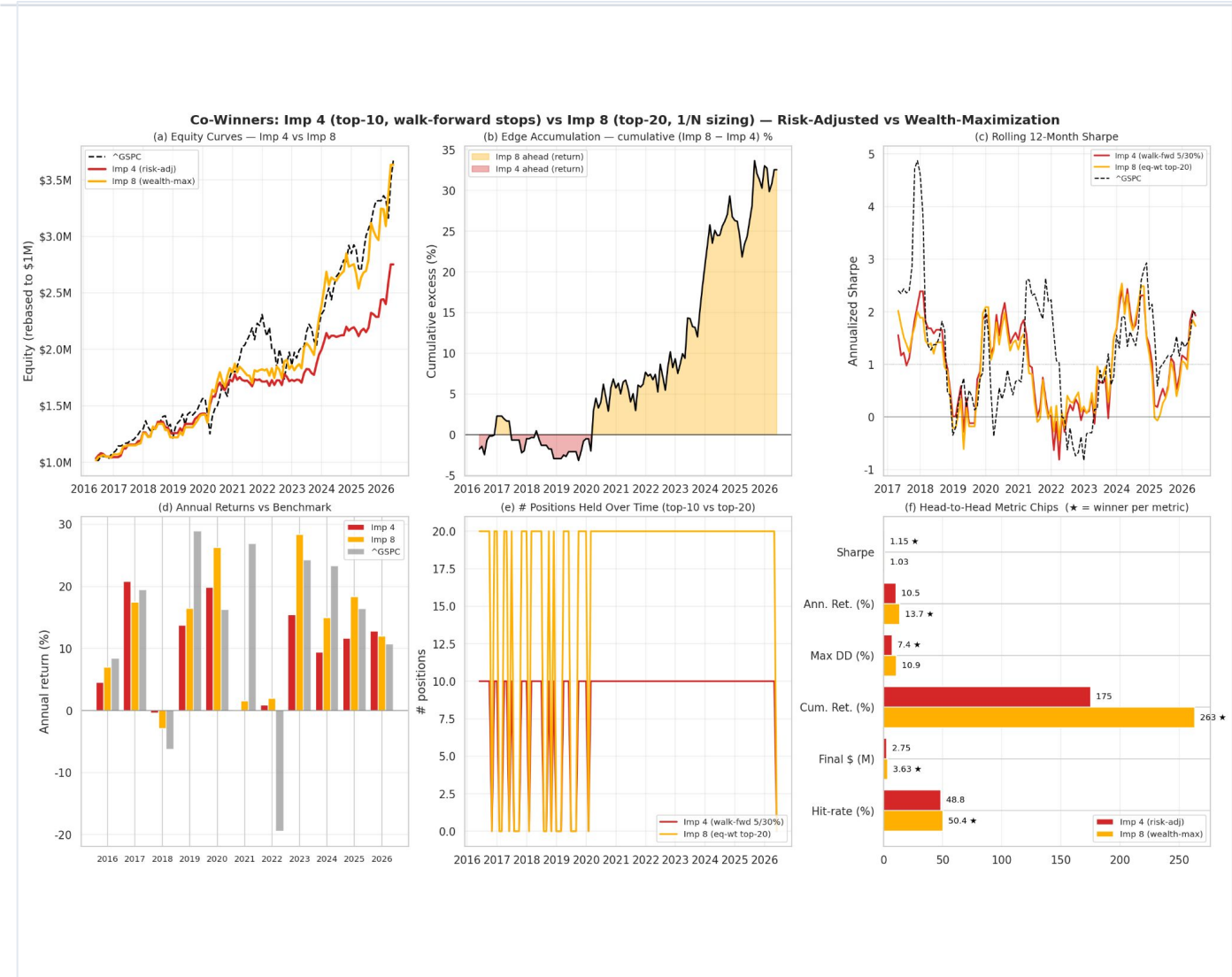
What this means

- Imp 6 is the more paper-faithful HZZ trend implementation.
- But in this long-only S&P 500 setup it trades more and earns less.
- Imp 4's assignment-style expanding ^GSPC trend has better coverage and lower churn.
- Both fail the final Romano-Wolf index test.

Do not present this as “the paper failed”; present it as “the paper-faithful adaptation did not dominate under this specific project design.”

Improved 4 vs Improved 8: risk-adjusted vs wealth-maximization

The project has two winners because the objective function changes.

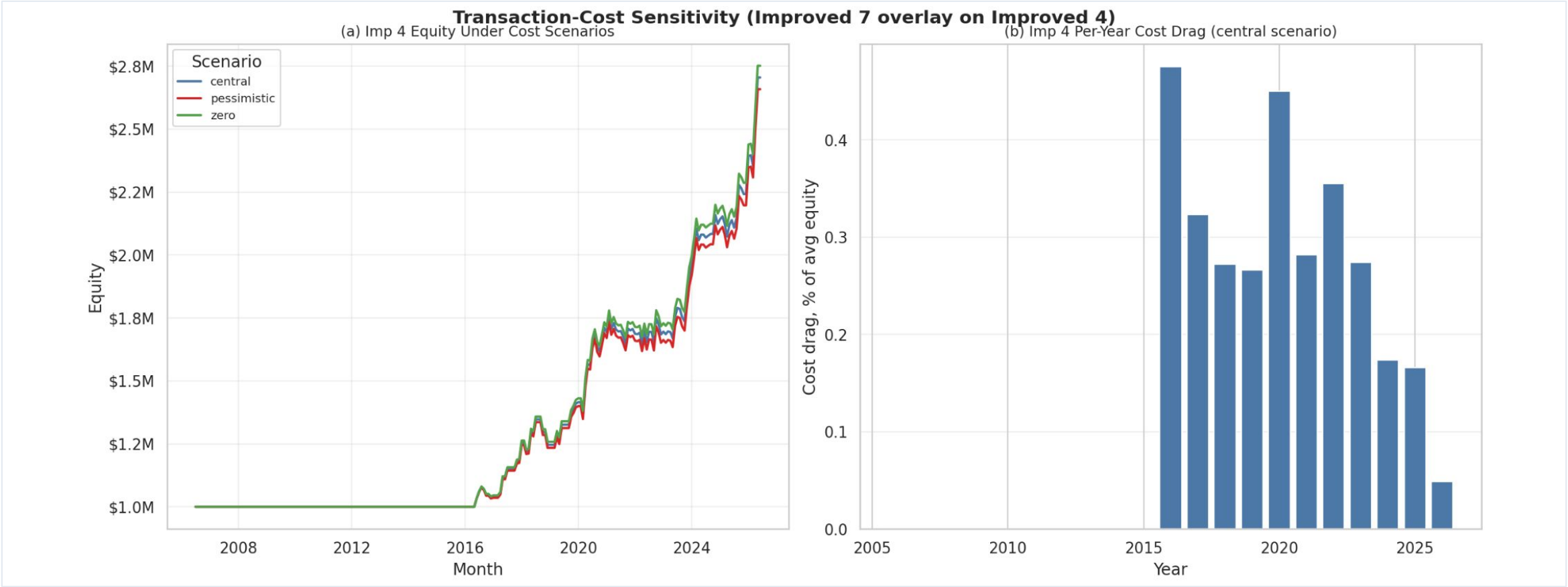


Metric	Imp 4	Imp 8	Winner
Sharpe	1.15	1.03	Imp 4
Annualized Return	10.48%	13.71%	Imp 8
Max Drawdown	7.4%	10.9%	Imp 4
Cumulative Return	175.2%	263.3%	Imp 8
Final Equity	\$2.75M	\$3.63M	Imp 8
Avg positions	8.4	16.9	-
Hit-rate	48.8%	50.4%	Imp 8
MC p-value raw	0.088	0.306	Imp 4
Romano-Wolf vs ^GSPC	No	No	Tie

Imp 4 is the risk-adjusted candidate. Imp 8 is the wealth candidate. Neither is statistically validated against ^GSPC after Romano-Wolf correction.

Transaction-cost sensitivity: Improved 7 overlay

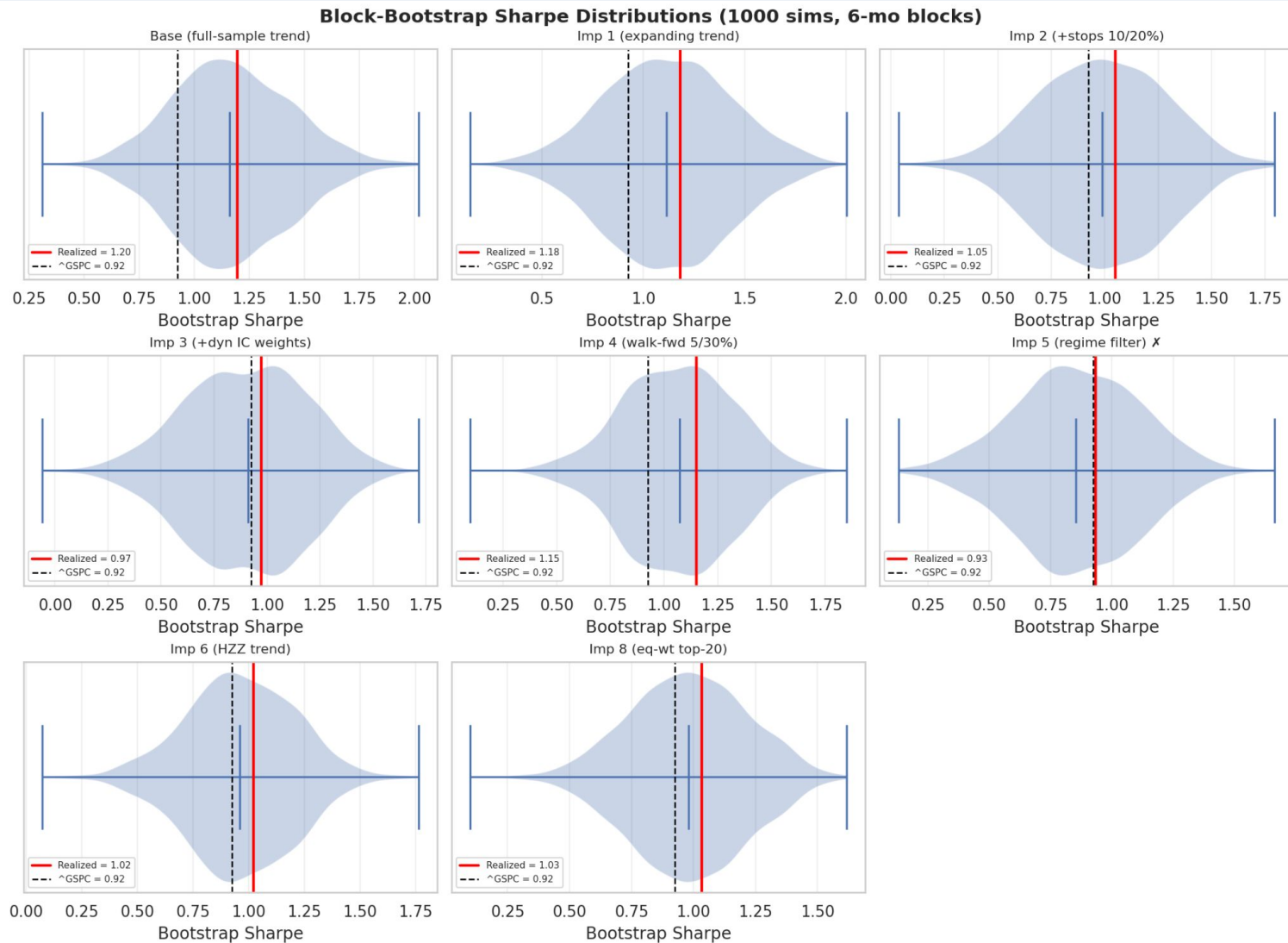
Improved 7 is a cost study on finalists, not a standalone strategy.



Central and pessimistic costs reduce Sharpe only modestly; Imp 4 remains ahead of Imp 6. Scope caveat: this is a vector cost sensitivity, not a full dated Backtrader commission model.

Block-bootstrap Sharpe distributions

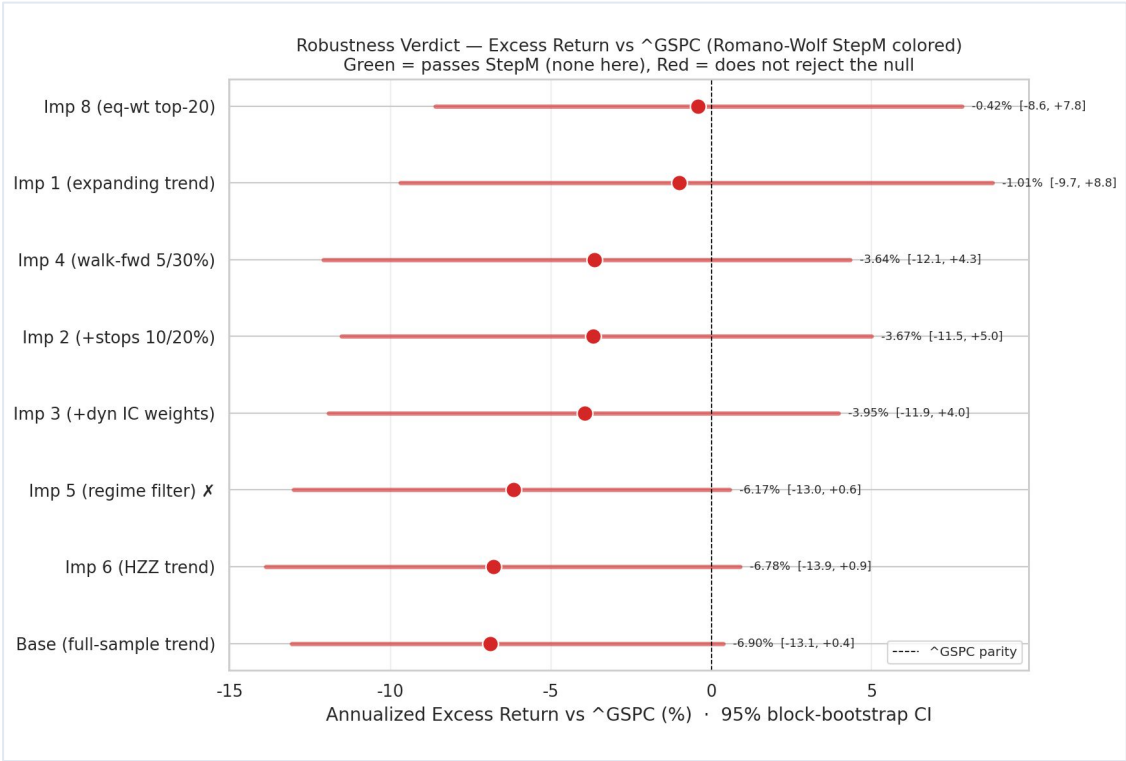
Raw Sharpe values are uncertain even before correcting for multiple strategy search.



Final robustness verdict: multiple-comparison correction

Hansen SPA and Romano-Wolf StepM control for trying many strategy variants.

Test	Statistic	Reject H0?
Hansen SPA overall	p_consistent = 0.6706	No
StepM - Imp 8	Ann. excess = -0.42%	No
StepM - Imp 1	Ann. excess = -1.01%	No
StepM - Imp 4	Ann. excess = -3.64%	No
StepM - Imp 2	Ann. excess = -3.67%	No
StepM - Imp 3	Ann. excess = -3.95%	No
StepM - Imp 5	Ann. excess = -6.17%	No
StepM - Imp 6	Ann. excess = -6.78%	No
StepM - Base*	Ann. excess = -6.90%	No



This is the most important result: attractive raw metrics do not survive the family-wise correction against ^GSPC.

Therefore the correct final claim is “promising research evidence,” not “an investable strategy that beats the market.”

Conclusion: would we invest?

Research-grade pipeline; not yet a production investment strategy.

Investment decision

- No, not as a production strategy today.
- Imp 4 and Imp 8 are useful research candidates, but neither survives Romano-Wolf vs $\wedge$ GSPC.
- The current-constituent S&P 500 universe creates survivorship bias.
- Main runs are zero-cost; cost tests are informative but not a complete execution model.
- A real deployment would need WRDS/CRSP point-in-time constituents, delisting returns, stricter out-of-sample testing and live paper trading.

Final claim: the project is valuable because it shows both attractive raw performance and why that evidence is not sufficient after rigorous correction.

Appendix: Codes

Core architecture: one hub, many focused runners

project_core.py contains the reusable implementation; each runner changes a narrowly defined experiment.

```
# project_core.py: central constants and output folders
CUTOFF = pd.Timestamp("2026-05-31")
EVALUATION_START = pd.Timestamp("2016-05-31")
BENCHMARK = "^GSPC"
INITIAL_CASH = 1_000_000.0
CASH_PER_TRADE = 100_000.0

BASE_RESULTS_DIR = RESULTS_DIR / "base_strategy"
IMPROVED_4_RESULTS_DIR = RESULTS_DIR / "improved_strategy_4"
IMPROVED_8_RESULTS_DIR = RESULTS_DIR / "improved_strategy_8"
ROBUSTNESS_RESULTS_DIR = RESULTS_DIR / "robustness"
```

Implementation logic

- All paths and strategy names live in one file, avoiding silent divergence across scripts.
- CUTOFF and EVALUATION_START are global, so every strategy is compared on the same window.
- Each runner writes to its own results folder; aggregation reads those saved files.

Data pipeline: frozen CSVs → monthly factor panel

The code is designed around reproducibility: raw inputs are stored, loaded, filtered, then transformed into a monthly panel.

```
def load_raw_data():
    prices = pd.read_csv(RAW_DIR / "sp500_prices_long.csv", ...)
    prices = prices[prices["date"] <= CUTOFF]

    daily = pd.read_csv(RAW_DIR / "sp500_fundamentals_daily_long.csv", ...)
    statements = pd.read_csv(RAW_DIR /
                             "sp500_fundamentals_statements_long.csv", ...)
    statements = statements[statements["date_available"] <= CUTOFF]

    constituents = pd.read_csv(RAW_DIR / "sp500_constituents.csv")
    index = download_or_load_index()
    return prices, daily, statements, constituents, index
```

StrategySpec: one object controls each strategy variant

```
@dataclass(frozen=True)
class StrategySpec:
    name: str
    weights: dict[str, float]
    top_n: int = 10
    regime_filter: bool = False
    stop_loss: float | None = None
    take_profit: float | None = None
    trend_col: str = "trend_z"
    weight_method: str = "static"
    sizing_method: str = "fixed_cash"
    sizing_target_pct: float = 0.10
```

Variant	Code-level change
Base	top_n=10; trend_col="trend_z"; no stop/take
Imp 1	trend_col="trend_expanding_z"; no look-ahead
Imp 2	adds 10% stop / 20% take
Imp 3	weight_method="rolling_rank_ic"
Imp 4	top-10; 5% stop / 30% take
Imp 5	Imp 4 + regime_filter=True
Imp 6	trend_col="trend_hzz_z"
Imp 8	top_n=20; percent_of_equity; 5% each

Vector simulator: monthly research engine

Vector simulation is used for strategy comparison, Monte Carlo, common-window metrics, and cost overlays.

```
def simulate_vector_strategy(panel, spec, cost_schedule=None):
    tmp = panel.copy()
    tmp["score"] = score_for_spec(tmp, spec)
    equity = INITIAL_CASH

    for month, g0 in tmp.groupby("month", sort=True):
        eligible = g0.loc[g0["eligible"] & g0["score"].notna()].copy()
        selected = select_positions_for_spec(eligible, spec, equity)
        selected["gross_stock_return"] = stop_take_return(
            selected, spec.stop_loss, spec.take_profit)
        selected["realized_stock_return"] = selected["gross_stock_return"] -
cost_drag
        pnl = per_position_dollars * selected["realized_stock_return"].sum()
        equity = equity + pnl
```

What the vector engine is for

- Scores are computed from the StrategySpec's chosen factor columns.
- A monthly groupby selects the top-N eligible stocks.
- Stop/take returns are approximated with next-month open/high/low/close.
- Costs enter as a round-trip return drag in the cost-sensitivity layer.
- metrics_over_evaluation_window restates every strategy from \$1M at 2016-05-31.

Vector backtest is the research engine; Backtrader is the physical broker simulation for execution realism.

Backtrader engine: daily execution and protective orders

Backtrader is used where order mechanics matter: entries, rebalances, stop-losses, take-profits and long-only checks.

```
# after a market entry fills, submit protective exits
stop = self.sell(data=data, size=int(size),
    exectype=bt.Order.Stop,
    price=entry_price * (1 - stop_loss))

limit = self.sell(data=data, size=int(size),
    exectype=bt.Order.Limit,
    price=entry_price * (1 + take_profit),
    oco=stop) # one-cancels-other

# separate sizers
FixedCashSizer(cash_per_trade=100_000)
EquityPercentSizer(target_pct=0.05) # Improved 8
```

Order lifecycle

- Monthly signals define the target ticker set.
- Daily adjusted OHLCV feeds determine whether protective exits are hit before the next rebalance.
- Protective orders are submitted after the entry fills, because the realized entry price is needed.
- OCO prevents both stop and take-profit from executing for the same position.
- Rebalance exits cancel protective orders; assertions check no short position is left behind.

Improved 4 code: stop/take grid selection

The 5% stop and 30% take-profit are selected from a constrained training-period grid, then rerun in Backtrader.

```
STOP_LOSSES = [0.05, 0.075, 0.10, 0.125, 0.15, 0.20]
TAKE_PROFITS = [0.10, 0.15, 0.20, 0.25, 0.30, 0.40]
TRAIN_END = pd.Timestamp("2020-12-31")

out["robust_train_score"] = (
    out["train_sharpe"]
    + out["train_max_drawdown"].fillna(0.0)
    - out["train_stability_gap"].abs().fillna(0.0)
)

eligible = grid[
    grid["train_total_return"].gt(0)
    & grid["train_max_drawdown"].gt(-0.25)
    & grid["neighbor_count"].ge(3)
]
selected = eligible.sort_values(
    ["robust_train_score", "train_sharpe"], ascending=False).iloc[0]
```

Why this is defensible

- The grid excludes take-profit values below the stop-loss.
- Selection uses training metrics only, not post-2020 test performance.
- The score penalizes isolated peaks by comparing each cell with neighbouring grid cells.
- Selected parameters and top grid rows are saved to CSV.
- The selected candidate is then run through daily Backtrader.

Improved 6 code: HZZ cross-sectional trend

This is the closest paper-faithful variant; the signal is stock-specific, not an index-level trend score.

```
def inject_hzz_trend(panel, monthly, prices):
    stock_ma = core.compute_stock_ma_signals(prices)
    ratios = core.stock_ma_ratios(stock_ma)

    betas = core.cross_sectional_trend_betas(monthly, ratios)
    smoothed = core.smooth_trend_betas(
        betas, window=core.HZZ_SMOOTH_WINDOW)
    trend_hzz = core.hzz_predicted_returns(ratios, smoothed)

    enriched = panel.merge(trend_hzz, on=["ticker", "month"], how="left")
    enriched["trend_hzz_z"] = core.winsorized_zscore_by_month(
        enriched, "trend_hzz_raw", "trend_hzz_z")
    return enriched, betas, smoothed
```

What is technically different

- For each month, regress next-month stock returns on 11 normalized MA ratios across the eligible cross-section.
- Betas are smoothed with a strict 12-month trailing window, excluding contemporaneous future-looking beta.
- Predicted returns become trend_hzz_raw, then monthly winsorized z-scores become trend_hzz_z.
- Imp 6 changes only trend_col; top-N, stop/take and fixed-cash sizing inherit Imp 4.

Improved 8 code: top-20 and 1/N equity sizing

The top-20 expansion and percent-of-equity sizing are mechanically coupled, so they are treated as one design change.

```
TOP_N = 20
SIZING_TARGET_PCT = 1.0 / TOP_N # 5% per position

def make_improved_8_spec(stop_loss, take_profit):
    return core.StrategySpec(
        name=core.IMPROVED_8_STRATEGY_NAME,
        weights={"roe": 1, "pe": 1, "momentum": 1, "trend": 1},
        top_n=TOP_N,
        trend_col="trend_expanding_z",
        stop_loss=stop_loss,
        take_profit=take_profit,
        sizing_method="percent_of_equity",
        sizing_target_pct=SIZING_TARGET_PCT,
    )
```

Why the code changes both fields

- \$100k fixed sizing cannot hold 20 names with \$1M starting capital.
- EquityPercentSizer targets 5% of current portfolio value per new position.
- This lets the strategy keep roughly 20 names as equity grows or shrinks.
- Result interpretation: higher terminal wealth, lower Sharpe and deeper drawdown.

Robustness code: aggregation, SPA and Romano-Wolf StepM

The final statistical verdict is generated after all strategy outputs are saved and aligned on the common evaluation window.

```
# run_multi_comparison_test.py
wide = load_strategy_returns()           # months x strategies
benchmark = load_benchmark_returns(wide.index)

benchmark_loss = -benchmark.values
model_losses = -wide.values

spa = SPA(benchmark_loss, model_losses,
          reps=10_000, block_size=6)
spa.compute()

stepm = StepM(benchmark_loss, model_losses,
              size=0.05, reps=10_000, block_size=6)
stepm.compute()
```

How the final “No” is produced

- aggregate_all_strategies.py first builds all-strategy metrics, walk-forward, benchmark and MC tables.
- The robustness script aligns strategy returns and ^GSPC returns month-by-month.
- Loss is negative return, so lower loss means better performance.
- Stationary block bootstrap uses 10,000 reps and 6-month blocks.
- SPA asks whether any strategy beats the benchmark after search correction; StepM asks which strategies survive individually.